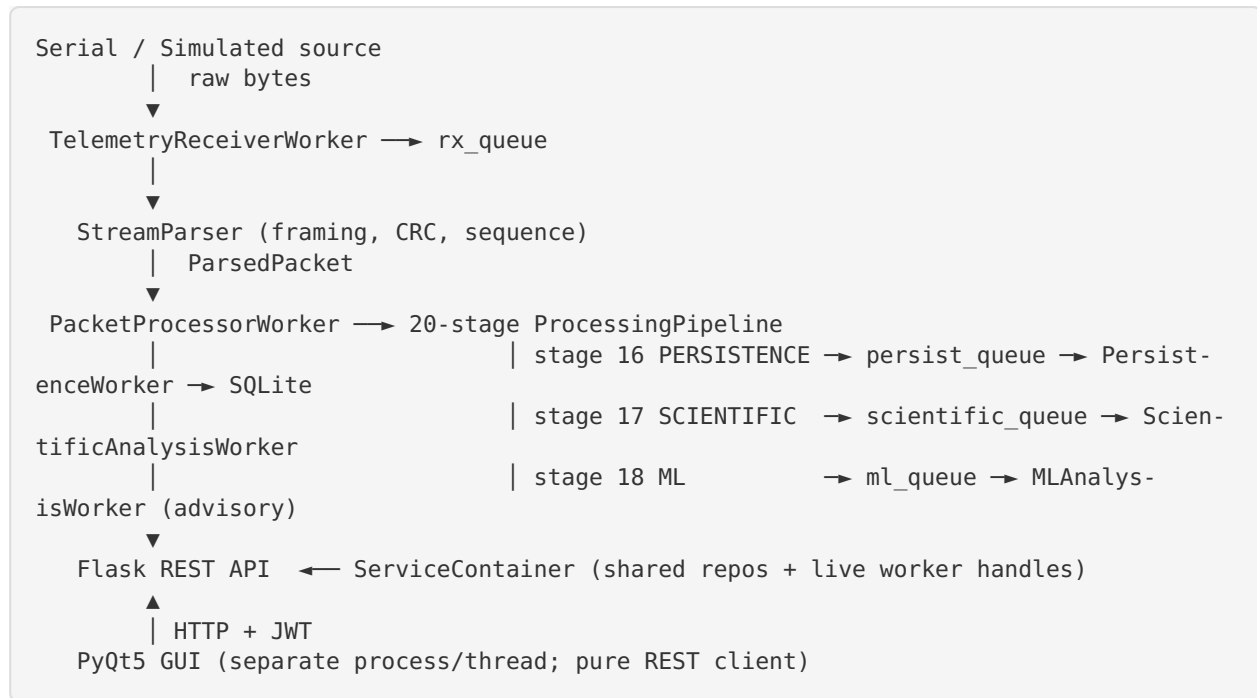


Engineering Manual

1. Architecture overview

AirOne V7.1 is a multi-threaded backend with a decoupled REST API and an optional GUI client. Data flows in one direction through explicit stages:



2. Capability tiers

The system is organised into four tiers with a strict rule: **a failure in a higher tier can never corrupt or disable a lower tier.**

Tier	Name	Contents	If it fails
1	Raw telemetry	Receive, parse, CRC, persist	Nothing else runs — this is the floor. Reported explicitly.
2	Science	37 deterministic scientific analyses	Tier 1 keeps capturing and storing raw data.
3	ML / AI	Advisory anomaly detection & drift	Tiers 1-2 unaffected; ML is advisory only.
4	Presentation	GUI, plots, judge mode	Backend fully unaffected; it is a REST client.

This ordering is enforced structurally: the ML tier holds **no reference** to the mission state machine (verified by `tests/test_ml.py`), and the GUI runs as a separate REST client.

3. The 20-stage pipeline

Each stage is an explicit `PipelineStage`. A stage failure is caught, the affected measurement is marked with the appropriate `QualityState`, and processing continues — one bad sensor never aborts the frame.

#	Stage	Responsibility
0	RECEIVE	Accept the decoded frame into the pipeline
1	AUTHENTICATE	Source/authenticity checks
2	DECODE	Decode payload fields
3	CRC_VALIDATE	Verify CRC32; mark <code>CORRUPTED</code> on mismatch
4	FEC_RECOVER	Reed-Solomon recovery if FEC present (optional dep)
5	SCHEMA_VALIDATE	Required-field presence; missing → <code>MISSING</code>
6	UNIT_NORMALIZE	Convert to canonical SI units
7	TIMESTAMP_VALIDATE	Clock-drift/ordering sanity
8	RANGE_CHECK	Physical range gating; out-of-range → <code>INVALID</code>
9	RATE_CHECK	Rate-of-change gating; implausible jump → <code>SUSPECT</code>
10	DUPLICATE_CHECK	Sequence-number duplicate detection
11	ORDER_CHECK	Out-of-order detection
12	SENSOR_QUALITY	Rolling per-sensor validity scoring
13	CALIBRATION	Apply calibration; tags <code>CALIBRATED</code> provenance
14	FILTERING	Kalman / EKF / moving-average filters (configurable)
15	SENSOR_FUSION	Multi-sensor fusion; tags <code>FUSED</code> provenance
16	PERSISTENCE	Enqueue to persistence worker
17	SCIENTIFIC	Enqueue to scientific worker

#	Stage	Responsibility
18	ML	Enqueue to ML worker (advisory)
19	VISUALIZATION	Prepare visualisation-ready snapshot

Stages can be restricted via `pipeline.enabled_stages` in config (null = all).

4. Workers

All workers derive from `BaseWorker` (a stoppable thread with a `health_status` dict). The `Orchestrator` builds the queues, wires the workers, and starts them in dependency order; shutdown is the reverse.

Worker	Role
<code>TelemetryReceiverWorker</code>	Reads raw bytes from serial/sim into <code>rx_queue</code>
<code>PacketProcessorWorker</code>	Parses packets and drives the pipeline
<code>PersistenceWorker</code>	Batches measurements into SQLite
<code>ScientificAnalysisWorker</code>	Runs the analysis registry over a rolling window
<code>MLAnalysisWorker</code>	Advisory anomaly detection over a rolling window
<code>HealthMonitorWorker</code>	Tracks queue depths and worker health

5. The ServiceContainer

`ServiceContainer` is the single shared-state object. It holds the database manager, all repositories, the mission state machine, and **live handles** to the scientific and ML workers (`scientific_worker` , `ml_worker`) so the API can surface live results. These handles are `None` until the orchestrator sets them, and every endpoint that depends on them returns an explicit `NOT_CONFIGURED` state when they are absent rather than inventing a result.

6. Extending the system

Adding a scientific analysis

Implement a function returning a `ScientificResult` and register it in `src/scientific/registry.py::build_default_registry` . It is automatically

exposed via `/api/v1/analysis`. Always populate `uncertainty` and state assumptions — see [scientific_methods.md](#) (`scientific_methods.md`).

Adding an anomaly detector

Subclass `BaseDetector` in `src/ml/detectors.py`, set `name / method`, implement `fit / score`, and add it to `DETECTOR_TYPES` in `src/ml/trainer.py`. Probe its optional dependency at import and set `available / reason` honestly so `/api/v1/ml/detectors` reflects reality.

Adding an API endpoint

Add a route to the relevant blueprint under `src/api/routes/`, protect it with `@jwt_required` and `@require_role(...)`, and return via `envelope(...)` so the response contract stays consistent.

7. Running the tests

```
cd airone
AIRONE_JWT_SECRET=test_secret_key_that_is_definitely_long_enough_123456 \
QT_QPA_PLATFORM=offscreen python -m pytest -q
```

See [test_plan.md](#) (`test_plan.md`) for what each test group proves.